

Lab 1 Deliverables

Charlie Caravati

3.2

```
>>> import numpy as np
>>> import matplotlib.pyplot as plt
>>> x=np.linspace(1,5,1)
>>> x
array([1.])
>>> x=np.linspace(1,5,5)
>>> x
array([1., 2., 3., 4., 5.])
>>> x=np.arange(1,5,1)
>>> x
array([1, 2, 3, 4])
>>> x=np.linspace(1,5,5)
>>> y=np.arange(1,6,1)
>>> x
array([1., 2., 3., 4., 5.])
>>> y
array([1, 2, 3, 4, 5])
>>> x[0]
np.float64(1.0)
>>> print("the frist three entries of x are", x[0], x[1], x[2])
the frist three entries of x are 1.0 2.0 3.0
>>> w=10*(-np.linspace(1,10,10))
>>> w
array([1.e-01, 1.e-02, 1.e-03, 1.e-04, 1.e-05, 1.e-06, 1.e-07, 1.e-08,
       1.e-09, 1.e-10])
>>> x=np.arange(1,11,1)
>>> x
array([ 1,  2,  3,  4,  5,  6,  7,  8,  9, 10])
>>> plt.semilogy(x,w)
[<matplotlib.lines.Line2D object at 0x107adc410>]
>>> plt.xlabel("x (linear)")
Text(0.5, 47.04444444444444, 'x (linear)')
>>> plt.ylabel("y (log scale)")
Text(64.57762803819443, 0.5, 'y (log scale)')
>>> plt.show()
>>> w3 = 3*w
>>> s=3*w
>>> plt.semilogy(x,w)
[<matplotlib.lines.Line2D object at 0x10e309950>]
>>> plt.semilogy(x,s)
[<matplotlib.lines.Line2D object at 0x10e309590>]
>>> plt.xlabel("x (linear)")
Text(0.5, 47.04444444444444, 'x (linear)')
>>> plt.ylabel("y (log)")
Text(64.57762803819443, 0.5, 'y (log)')
>>> plt.show()
```

4.2.1

```
import numpy as np
import numpy.linalg as la
import math

def driver():

    n = 2
    x = np.linspace(0,np.pi,n)

    # this is a function handle. You can use it to define
    # functions instead of using a subroutine like you
    # have to in a true low level language.
    f = lambda x: x**2 + 4*x + 2*np.exp(x)
    g = lambda x: 6*x**3 + 2*np.sin(x)

    y = np.array([0,1])
    w = np.array([1,0])

    # evaluate the dot product of y and w
    dp = dotProduct(y,w,n)

    # print the output
    print('the dot product is : ', dp)

    return

def dotProduct(x,y,n):
    # Computes the dot product of the n x 1 vectors x and y
    dp = 0.
    for j in range(n):
        dp = dp + x[j]*y[j]

    return dp

driver()
```

4.2.2

```
"""
This is a modified matrix multiplication code
"""
#####

import numpy as np
import numpy.linalg as la
import math

def driver():
```

```

    n = 2
    y = np.array([[1,2],[3,4]])
    w = np.array([[5,6],[7,8]])

# evaluate the matrix multiplication of y and w
    mp = matmul(y,w,n)

# print the output
    print('the matrix product is : ', mp)

    return

def matmul(y,w,n):
#   Computes the matrix multiplication of matrices y, w

    rowsY, columnsY = y.shape
    rowsW, columnsW = w.shape

    mp = np.zeros((rowsY, columnsW))

    for i in range(rowsY):
        for j in range(columnsW):
            for k in range(columnsY):
                mp[i, j] = mp[i, j] + y[i, k] * w[k, j]

    return mp

driver()

```

```

>>> Lab1 % python3 testMat.py
the matrix product is :  [[19. 22.]
 [43. 50.]]

```

4.2.3

```

>>> y = np.array([[1,2],[3,4]])
>>> w = np.array([[5,6],[7,8]])
>>> y@w
array([[19, 22],
 [43, 50]])

>>> y = np.array([1,2])
>>> w = np.array([3,4])
>>> np.dot(y,w)
np.int64(11)

```

The built in commands are much faster.

